

Video 1 — Demo bộ automation

23127195 · FR-09 Mã giảm giá · mục tiêu 6 phút (đề yêu cầu ≥ 5) Quay face-cam trên máy tính → **không cần** `whoami` / `hostname` .

Chuẩn bị

```
cd D:\Kiem_thu\HW4\HW04-TESTING
node eshop-sut/backend/database.js
```

Hai terminal nền: `node server.js` (backend) · `npm run dev` (frontend-web)

- ☐ `http://localhost:5173` lên được
 - ☐ Mở sẵn: **Terminal** · **VS Code** (`tests/fr09-coupon.spec.js`) · **Chrome** (`/checkout`)
 - ☐ Bật webcam, phóng to cỡ chữ, tắt thông báo Windows
-

1 · Mở đầu — 30 giây

Face-cam:

Em chào thầy cô. Em là **Trần Minh Hùng, MSSV 23127195**, bài HW04 Automation Testing.

Video này em demo bộ test tự động cho **FR-09 — Mã giảm giá**: chạy trên ba trình duyệt, xem báo cáo HTML, và kể lại **một lỗi em đã sửa** trong script do AI sinh ra.

2 · Giới thiệu bộ test — 1 phút

Màn hình: VS Code → mở `tests/data/fr09-coupon-calculations.csv`

Hệ thống kiểm thử là **EShop**. Em chọn ba feature từ HW02, tổng **62 test case**, chạy trên ba engine — **186 lượt chạy, 9 browser run**.

Đề yêu cầu **data-driven**: dữ liệu phải để file riêng, không viết cứng trong code. Đây là file CSV chứa mười trường hợp tính giảm giá.

(Trở dòng TC-01)

Ví dụ: mã SAVE10 giảm 10% trên đơn 500 nghìn thì phải giảm 50 nghìn, còn trả 450 nghìn. Đây là con số **theo đặc tả**, không phải theo cách hệ thống đang chạy.

Mở `tests/fr09-coupon.spec.js`

File spec có 18 test case. Đầu file em ghi rõ **sáu kiểu assertion**, đề chỉ yêu cầu tối thiểu ba.

Test nào có nhãn `@bug` là em **cố ý để fail** — chúng kiểm tra theo đặc tả, còn hệ thống thì đang sai. Mỗi test đơ ứng với một lỗi thật.

3 · Chạy 3 trình duyệt — 2 phút ⚠ **BẮT BUỘC**

```
node scripts/run-multibrowser.mjs tests/fr09-coupon.spec.js
```

Nói trong lúc chờ (~2 phút):

Script chạy lần lượt Chromium, Firefox, WebKit — mỗi engine sinh **một báo cáo HTML riêng**.

Em phải viết script riêng vì nếu chạy `npx playwright test` thẳng thì Playwright gộp cả ba vào một báo cáo, trong khi đề yêu cầu mỗi browser run một report.

(Chromium xong)

Chromium: **12 pass, 6 fail** — sáu fail chính là các test `@bug`.

(Firefox đang chạy)

Ba engine này khác nhau về bản chất: Chromium dùng nhân Blink, Firefox dùng Gecko, WebKit là nhân Safari. Playwright tải sẵn cả ba về máy và điều khiển qua WebSocket — **trình duyệt thật, không phải giả lập.**

(Bảng tổng kết)

Cả ba đều **12 pass, 6 fail** giống hệt nhau. Sự đồng nhất này chứng tỏ bộ test **không flaky.**

4 · Mở HTML report — 1 phút ⚠ **BẮT BUỘC**

```
npx playwright show-report playwright-report/fr09-coupon-chromium
```

(Trở vào tiêu đề, dừng 3 giây)

Xin thầy cô chú ý dòng tiêu đề: **Run by 23127195** kèm **timestamp ISO** của đúng lần chạy vừa rồi. Đây là yêu cầu chống gian lận — nhúng vào lúc chạy, không thể thêm sau.

(Click test TC-01 fail)

Test TC-01 fail: kỳ vọng giảm **50 nghìn**, thực tế nhận **âm 4 triệu 500 nghìn.**

5 · Chứng minh bug là thật — 45 giây

Chrome → `http://localhost:5173/checkout`

(Điền Tổng tiền `500000` · nhập `SAVE10` · bấm Áp dụng)

Em làm lại bằng tay để chứng minh đây là lỗi thật của hệ thống, không phải test viết sai.

(Trở kết quả)

Giao diện hiện tích xanh "Áp dụng thành công, giảm 10%". Nhưng nhìn xuống: **Tiết kiệm âm 4 triệu 500 nghìn, Thành tiền 5 triệu**. Đơn 500 nghìn mà khách trả 5 triệu — **gấp 10 lần**.

Nguyên nhân: backend viết công thức là $\text{tổng} \times (1 - \text{giá trị giảm})$. Với mã 10% thì $1 - 10 = -9$, nên số tiền giảm thành **số âm**; mà thành tiền = tổng trừ số giảm, trừ số âm thành cộng. Công thức đúng phải là $\text{tổng} \times \text{phần trăm} \div 100$.

Đây là lỗi nghiêm trọng nhất em tìm được, đã ghi thành **Issue số 7**.

6 · Lỗi em đã sửa trong script AI — 1 phút 15 ⚠ **BẮT BUỘC**

Phần đề bắt buộc. Nói kỹ, đừng vội.

Face-cam (hoặc VS Code mở `tests/fr01-register.spec.js` phần comment R-03)

Cuối cùng em xin kể một lỗi em đã phát hiện và sửa trong code do AI sinh. Đây là lỗi nguy hiểm nhất vì nó **không làm test đỏ — nó làm test xanh giả**.

Ban đầu, để chứng minh "email sai định dạng thì không tạo được tài khoản", AI viết: sau khi bấm Đăng ký thì kiểm tra URL có còn ở `/register` không. Nếu còn thì coi như hệ thống đã từ chối.

Nghe rất hợp lý. Và **bốn test đó đều báo xanh**.

Nhưng khi em thử bằng tay thì hệ thống **thực tế chấp nhận** email `abc` — tức bốn test kia đáng lẽ phải đỏ.

Em truy nguyên và hiểu ra: đây là **single-page application**. Ngay sau khi bấm nút, request lên server **chưa xử lý xong**, nên URL đương nhiên vẫn là `/register` — bất kể server quyết định gì. Câu lệnh kiểm tra khớp **ngay lập tức** và chuyển xanh trước khi ứng dụng kịp quyết định.

Nói cách khác: **bộ test đã che giấu đúng cái lỗi mà nó được viết ra để tìm**.

Em sửa bằng cách bỏ hẳn việc đoán qua URL, thay bằng **hỏi thẳng cơ sở dữ liệu**: còn bao nhiêu dòng người dùng mang email này? Từ chối đúng thì phải là 0. Sau khi sửa, cả bốn test chuyển đỏ — đó mới là kết quả phản ánh đúng chất lượng hệ thống.

Bài học: **một test xanh không chứng minh phần mềm đúng, nó có thể chỉ chứng minh câu lệnh kiểm tra đặt sai chỗ**.

7 · Kết — 20 giây

Chrome → GitHub Issues

Tổng kết: **3 feature, 62 test case, 9 browser run, 15 lỗi** đã đăng đầy đủ lên GitHub Issues kèm ảnh bằng chứng.

Em cảm ơn thầy cô đã theo dõi.

Kiểm tra sau khi quay

- ☐ Video ≥ **5 phút**, có mặt bạn (face-cam) và giọng bạn
- ☐ Có cảnh chạy **3 trình duyệt**
- ☐ Report hiện "**Run by: 23127195**" + ISO timestamp, đọc được rõ
- ☐ Có đoạn kể **lỗi đã sửa**
- ☐ YouTube → **Unlisted** → dán link vào `README.md`